



A dual-stage deep learning approach for robust detection and identification of hardware trojans using monte-carlo dropout

Arash Golabi¹ · Abdelkarim Erradi¹ · Ahmed Bensaid¹ · Abdulla Al-Ali¹ · Uvais Qidwai¹

Published online: 25 May 2025
© The Author(s) 2025

Abstract

Hardware Trojans (HTs) pose a significant threat to the integrity and security of integrated circuits, particularly in critical systems where stealthy hardware modifications can lead to catastrophic consequences. Detecting such Trojans through side-channel analysis (SCA) remains a major challenge due to subtle signal variations and environmental noise. This paper presents a dual-path deep learning framework for the detection and identification of HTs via side-channel analysis. The proposed approach transforms time-series side-channel data- including power consumption, electromagnetic emissions, and timing information-into two distinct image-based representations using Markov Transition Fields (MTF) and a reshaping technique. These transformed representations feed into a two-stage architecture: an Attack Detector, which determines the presence of an HT, and an Attack Identifier, which classifies the specific type of detected HT. To enhance reliability, Monte Carlo Dropout (MCD) is integrated for uncertainty estimation, enabling the framework to flag low-confidence detections and support more robust decision-making. The proposed method is evaluated on publicly available AES hardware Trojan datasets from TrustHub and IEEE Dataport, demonstrating superior accuracy over existing approaches, particularly for complex HT variants with subtle attack signatures. Furthermore, the robustness of the proposed method has been assessed by introducing noise into the validation dataset to simulate real-world operational perturbations such as process variation, aging, and voltage level variations. The results demonstrate the effectiveness and applicability of the dual-path deep learning framework for hardware Trojan detection.

Keywords Hardware trojan detection · Time series analysis · Markov transition field · Convolutional neural networks · Deep neural networks.

This work was made possible by NPRP grant NPRP14S-0413-210206 from the Qatar National Research Fund (a member of the Qatar Foundation). The statements made herein are solely the responsibility of the authors. Open Access funding provided by the Qatar National Library.

✉ Arash Golabi
arash.golabi@qu.edu.qa

Abdelkarim Erradi
erradi@qu.edu.qa

Ahmed Bensaid
abensaid@qu.edu.qa

Abdulla Al-Ali
abdulla.alali@qu.edu.qa

Uvais Qidwai
uqidwai@qu.edu.qa

¹ Department of Computer Science and Engineering, Qatar University, Doha, Qatar

1 Introduction

Hardware Trojans (HTs) have emerged as critical security threats, compromising the reliability and security of electronic systems due to increasing globalization and the outsourcing of semiconductor manufacturing. Existing HT detection methods, although promising, often fail under realistic conditions characterized by process variation, environmental noise, and subtle malicious modifications. Therefore, developing robust and reliable HT detection solutions that address these limitations is an urgent and significant research challenge.

HTs are malicious modifications introduced into electronic circuits or designs, leading to unintended behavior of an integrated circuit (IC) or chip [1]. These Trojans can be stealthily embedded into a chip's design or manufacturing process, making detection a significant challenge. They are meticulously engineered to remain dormant, activating

only under specific conditions to evade detection and maintain normal operation in most scenarios [2]. HTs can disrupt normal operations, leak confidential information, and provide backdoor access to sensitive systems, potentially leading to catastrophic consequences, including data loss, financial devastation, or threats to human life, especially in critical infrastructures [3, 4].

HT attacks are sophisticated, evade traditional testing methods, and pose significant risks, making them a key concern for cybersecurity experts. The growing threat of HTs underscores the need for robust security measures to protect hardware from such vulnerabilities. Detecting HTs and mitigating their impact are essential for developing next-generation defenses for electronics. Continued research and development in this area are imperative to prevent the potentially severe consequences of HTs and ensure the integrity of technological infrastructure [5]. A primary approach to detecting HT attacks is Side-Channel Analysis (SCA), which involves monitoring physical parameters such as power consumption, electromagnetic emissions, timing intervals, and acoustic signals while the hardware is in operation. Unexpected deviations in these physical parameters may signal the presence of HTs, highlighting potential malicious alterations. However, these methods may face challenges due to noise and variations in normal operation, leading to false positives or negatives [6].

Integrating machine learning with SCA can substantially improve HT detection by analyzing side-channel signals generated by electronic devices [7]. The machine learning algorithms create models that capture standard device behaviors, enabling the identification of anomalies indicative of HTs. Traditional methods, such as Multilayer Perceptron (MLP), struggle with the high dimensionality and noise in side-channel signals, leading to decreased detection accuracy [8]. Using Convolutional Neural Networks (CNNs) for HT detection presents considerable benefits. CNNs excel at identifying subtle patterns in noisy datasets, improving HT detection sensitivity and precision. Their ability to filter out noise and focus on key features makes them particularly effective for minimizing false positives and reliably identifying hardware anomalies [9–11].

While 1D CNNs are commonly used for processing time series data, 2D CNNs offer significant advantages by transforming side-channel data into image-like representations and using their strong spatial pattern recognition capabilities. The Markov Transition Field (MTF) transformation converts time series data into a matrix format that visually resembles an image, bridging the gap between temporal data and CNN's image-centric processing capabilities [12, 13]. This transformation captures transition probabilities, preserving dynamic characteristics and temporal dependencies crucial for accurate HT detection. The MTF-transformed representation enhances CNN's ability to detect subtle anomalies,

such as transition patterns and hidden dependencies that are often less discernible in the original format [14]. Therefore, the integration of MTFs and CNNs significantly increases hardware security, particularly in the HT detection domain.

In the context of HT detection, ensuring high confidence in detection outcomes is crucial, especially in scenarios where environmental noise or hardware aging introduces variability in the data. To address these challenges, Monte-Carlo Dropout (MCD) is employed to quantify uncertainty in model predictions by using dropout as an approximation of Bayesian inference [15]. Incorporating MCD during inference allows the model to perform multiple stochastic forward passes, generating a distribution of predictions for each input. This approach not only produces a final prediction but also estimates associated uncertainty, providing a measure of confidence in HT detection. By assessing the standard deviation of predictions, MCD helps identify high-uncertainty cases, flagging them for further analysis, which results in a more robust detection system and ultimately improves the reliability and effectiveness of HT detection.

The primary contribution of this work is the introduction of a dual-stage deep learning architecture for detecting and identifying HTs using SCA and time-series imaging, with MCD to enhance prediction confidence and reliability. The approach transforms side-channel time series data into two image-like representations using MTF and reshaping techniques, which are processed through two separate CNN paths to maximize feature extraction. This dual-path setup forms the foundation of the dual-stage deep learning structure, consisting of an Attack Detector and an Attack Identifier. The Attack Detector determines the presence of HTs, while the Attack Identifier classifies the detected HTs. The integration of MCD enables the model to quantify uncertainty, particularly in scenarios with noise or aging effects that could reduce detection accuracy. This ensures a robust and reliable detection system by flagging uncertain cases for further analysis. The proposed method's effectiveness is demonstrated through simulations using publicly available datasets, showing superior performance compared to existing methods. Furthermore, the impact of noise is analyzed to provide insights into the robustness of the method under various operational conditions.

The structure of this article is organized as follows. Section 2 presents the background to HT attacks and provides an overview of existing HT detection methods. Section 3 summarizes related work and highlights key gaps in existing HT detection methods. In Section 4, the proposed methodology is outlined, with a focus on imaging time series data and employing MCD method. The results and discussion are elaborated in Section 5, where the effectiveness of the proposed approach is analyzed in detail. Finally, Section 6 concludes the paper with a summary of the key findings and implications of this study.

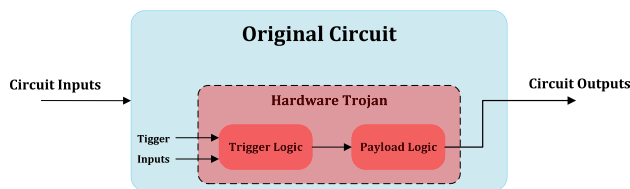


Fig. 1 General configuration of hardware Trojan [17]

2 Background and threat model

The importance of understanding HT attacks lies in their serious threat to the security and reliability of electronic systems, including critical infrastructure and consumer electronics, leading to serious risks that may result in catastrophic failures and breaches of privacy. To identify and neutralize these Trojans before they cause harm, effective detection methods must be developed. In an increasingly interconnected world, this will ensure hardware systems' reliability and security. This section provides a comprehensive background for understanding HT attacks and their detection methods.

Here, we provide an overview the structure and functioning of embedded HTs. Figure 1 presents a simplified block diagram of an embedded HT, illustrating the trigger and payload mechanism. HTs can be classified according to their activation triggers and malicious payloads [16]. Trigger mechanism refers to the condition or set of conditions that trigger the HT. The triggers can be highly specific, designed to remain hidden and undetected until a particular event occurs. The triggers vary greatly, ranging from simple events like a specific date or time to complex patterns of activity and specific inputs that may occur only in rare instances. The sophisticated trigger mechanism allows the Trojan to avoid detection during routine testing and operational use. This ensures it remains inactive until it can achieve its intended effect. Once activated, the payload of a HT is the actual malicious action carried out by the Trojan. Depending on the payload, a hardware Trojan can alter device functionality, leak sensitive information, degrade system performance, or cause physical damage.

As shown in Figure 2, HTs can be introduced at various stages of a chip's lifecycle, exploiting vulnerabilities from the initial design phase through to fabrication and final deployment [3]. During the design phase, HTs can be covertly introduced by tampering with third-party Intellectual Property (IP) cores, commonly used in chip design to reduce development time and costs. Additionally, HTs can compromise chip design software tools, inserting malicious code or circuitry that remains inactive until triggered by specific conditions. The fabrication phase is also susceptible to HTs, particularly when manufacturing is outsourced to untrusted facilities. These facilities can covertly modify chip layouts or tamper with the masks used in the lithography pro-

cess, embedding HTs directly into the silicon. This global outsourcing model, though economically beneficial, allows for subtle and undetectable alterations due to the microscopic scale and complexity of modern integrated circuits. Additional vulnerabilities can arise during the assembly and packaging stages. These stages offer further opportunities for HT insertion, either directly altering the silicon die or by incorporating malicious components within the chip package. Moreover, as chips move through the supply chain to reach assemblers or end-users, there are opportunities for interception and tampering, adding another layer of risk.

Due to the complexity of these threats, advanced security measures must be taken throughout the lifecycle of a chip. A comprehensive strategy is necessary to mitigate HT risks and ensure electronic systems' security and functionality. This strategy should include secure and verified design techniques, integrity assurance in manufacturing and assembly processes, and supply chain protection. Furthermore, developing and implementing sophisticated detection methods is vital for identifying and preventing Trojans that evade initial preventative measures. This ensures electronic systems' security and functionality. A reliable HT attack detection mechanism is essential since HT attacks can seriously compromise critical electronic infrastructure without effective detection methods.

3 Related works

This section provides a review of existing techniques for HT detection, including traditional methods, machine learning approaches, and deep learning-based solutions. While previous works have made significant progress in identifying HTs, challenges remain in achieving robustness against real-world perturbations and enhancing classification accuracy. We summarize key advancements in HT detection, analyze the limitations of current approaches, and identify the gaps that motivate further research. These insights provide as the foundation for our work, which addresses these challenges through a dual-stage deep learning framework with MCD for improved detection reliability.

Detecting HTs is challenging due to their low activation probability, significant variability, and their ability to mimic normal circuit behavior [18]. HT detection methods are divided into pre-silicon and post-silicon approaches [19, 20]. Pre-silicon methods focus on design verification and secure design to prevent Trojan insertion, while post-silicon methods identify HTs in manufactured chips using destructive (e.g., reverse engineering) or non-destructive (e.g., SCA, functional testing) techniques [21, 22]. Destructive methods, like reverse engineering, physically deconstruct devices to inspect for HTs, but are costly and impractical for large-scale testing. Non-destructive methods, such as logic testing

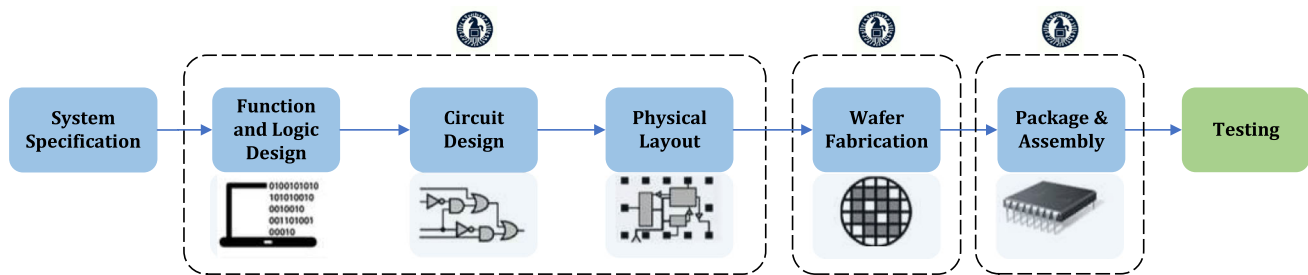


Fig. 2 An overview of IC design and manufacturing process, highlighting potential points for HT insertion

and SCA, allow for examining chips without damage, making them more suitable for widespread use [23]. Logic testing involves verifying the IC's logical integrity by comparing its functionality to its intended design [22]. This process involves stimulating the chip with various input conditions and observing the outputs to identify abnormal behavior. However, logic testing cannot detect dormant Trojans designed to activate under specific or rare conditions [24]. Such Trojans may remain undetected during standard testing procedures.

SCA is a non-intrusive technique used to detect HTs, particularly those that remain dormant during standard tests. SCA identifies anomalies by monitoring unintended signals such as power consumption and electromagnetic emissions, providing a crucial tool for maintaining hardware integrity across various applications, including consumer electronics and critical infrastructure [25, 26]. Due to its ability to detect Trojans that remain inactive during standard logic testing, this method is particularly useful [27]. SCA identifies potential anomalies associated with HTs by analyzing the device's behavior under different conditions and comparing it against a reliable baseline. Therefore, SCA forms an integral part of modern hardware security protocols, particularly for detecting Trojans designed to evade conventional logic tests. However, traditional SCA methods can be challenged by noise and variations in normal operation, which can lead to false positives or negatives [28].

Integrating machine learning with SCA significantly enhances the detection of HTs by enabling the analysis of large datasets, identifying subtle anomalies, and improving accuracy and efficiency in distinguishing normal from malicious behavior [7]. In this integrated methodology, machine learning techniques interpret side-channel data from electronic devices by developing complex mathematical representations that capture normal operational patterns. This representation allows for the precise detection and classification of deviations in side-channel emissions, which frequently indicate the presence of HTs, thus enhancing Trojan identification accuracy through the use of machine learning [29]. Recent review studies highlight the critical role of machine learning in detecting and securing HTs,

emphasizing its effectiveness in analyzing hardware security vulnerabilities, detecting complex security threats, and integrating into security strategies such as [30, 31].

Various machine learning techniques, both supervised and unsupervised, have been successfully applied to detect HT attacks by analyzing side-channel signals [32, 33]. Supervised methods such as Support Vector Machine (SVM) [34], Naïve Bayes [35], Random Forest [36], and XGBoost [37] have demonstrated effectiveness in identifying anomalies indicative of HTs. Recent work by [38] explores machine learning-assisted power-side channel analysis, enhancing classification accuracy across different Trojan states. Another approach proposed in [39] utilizes ensemble learning, combining multiple weak classifiers to detect HTs efficiently, achieving high performance across multiple TrustHub datasets. Moreover, innovative techniques like Deep Belief Networks (DBN) optimized with Stacked Autoencoder Hashing Optimization [40] and deep transfer learning [41] represent state-of-the-art approaches that enhance detection accuracy and adaptability in the field of HT attack identification. Deep learning architectures, particularly CNNs, have been widely adopted for HT detection due to their ability to extract complex patterns from side-channel signals. An attention mechanism has been incorporated into CNNs in [42], improving feature extraction efficiency. Recent research demonstrates the effectiveness of ResNeXt networks in Trojan detection [9], using hierarchical feature learning for multi-Trojan detection scenarios. The combination of SCA and CNN enhances feature extraction and provides robustness against noise, making it an effective tool for detecting HTs. By converting time-series signals into image-based representations, CNNs efficiently handle high-dimensional data, reducing the need for manual feature engineering and providing a scalable solution for modern cybersecurity challenges [43, 44].

These efforts demonstrate a broad strategy for addressing hardware security threats by applying machine learning and deep learning methods to create advanced and increasingly efficient HT detection mechanisms. Despite these advancements, several challenges remain. Many existing approaches lack uncertainty quantification, making them susceptible to

misclassification in the presence of real-world noise. Another major concern is the robustness of machine learning models under adversarial conditions, as slight perturbations in side channel signal can significantly impact classification performance. Our work addresses these challenges by introducing a dual-path CNN framework that integrates MCD method for uncertainty quantification, enhancing model reliability. We also employ a hybrid feature extraction approach, combining MTF with side-channel analysis, which improves robustness against process variations. This approach improves detection accuracy while offering a high-performance solution.

4 Hardware trojan detection pipeline: A dual-stage deep learning framework

This section first explains the proposed methodology designed to detect and identify potential HT threats. It begins by describing how time series data is transformed into a useful representation for analysis using MTF and reshaping techniques transformation. These transformations produce two separate 2D images from the time series data, enabling the effective application of deep learning models like CNNs to analyze the data by utilizing their spatial feature learning capabilities on these image representations. The extracted features are then passed through a classifier component, which determines whether an HT attack is present or not. This forms the Attack Detector and Attack Identifier stage. If an attack is detected, the model proceeds to the next stage, which classifies the specific Trojan attack type. Following the explanation of the proposed methodology, the algorithm for training and testing the two-stage detector and identifier model is described. Additionally, details about the dataset used for this study, consisting of power side-channel signals with both disabled and triggered hardware Trojans, are provided.

4.1 Image-based analysis of time series data

Transforming time series data into image-like formats has shown significant performance improvements in a variety of fields, enabling the application of CNNs for effective time series analysis by exploiting their strength in spatial feature recognition [45–48].

One method for transforming time-series data into an image-like format involves organizing the linear sequence of data points into rows and columns to form an image, named the reshaping technique. The time series data can then be transformed into a 2D matrix using a function that arranges the data row-wise into the specified dimensions. Given a time series, the reshaping process results in a matrix where the data points are systematically placed into a grid, preserving their sequential order. It should be noted that if the time

series length is not perfectly square, or if a different size matrix is desired, several alternatives are available [49, 50]. Padding can be applied by adding zeros to the end of the time series until its length is a perfect square. Alternatively, replication padding can be used by repeating the last value until the required length is achieved. Truncating the time series is another option, where extra data points are cut off if the time series is longer than needed. Interpolation can also be employed to adjust the time series length to match the desired dimensions. This involves using methods such as linear interpolation to increase or decrease the length of the time series to exactly fit the required number of points.

Another innovative method for this transformation is the use of MTF imaging. MTF imaging is a technique that converts time series data into a matrix of transition probabilities. This captures the dynamics and dependencies between different states of the data [51]. Traditional CNN models for side-channel analysis often rely on a single representation of time-series data, limiting their ability to capture both local and global dependencies. Our dual-path design mitigates this limitation by employing two transformation techniques: reshaping and MTF-based encoding. The reshaped representation effectively detects abrupt changes in the power consumption traces, which are key indicators of Trojan activation, while the MTF representation encodes underlying cyclic dependencies. By combining these complementary features, our model learns more discriminative patterns, improving classification accuracy compared to conventional CNN-based approaches. This matrix visualizes the likelihood of transitioning from one quantized state to another, providing a rich, visual representation of temporal dynamics [52]. The transformation into a transition probability matrix allows for the complex temporal patterns of time series signals to be visualized and analyzed in two-dimensional space. The MTF approach is particularly advantageous as it encodes both temporal changes and state dependencies into spatial patterns, facilitating the use of two-dimensional CNNs [53]. These CNNs excel at interpreting spatial relationships and patterns within the data, thereby enhancing the model's ability to uncover and analyze latent features and intricate patterns. The use of MTF imaging has been proposed in several studies, demonstrating its effectiveness in different domains, such as fault detection, financial forecasting, and stock market analysis [54–57].

4.1.1 Applying MTF for time series data analysis

MTF represents the time series as a transition probability matrix. The process involves calculating the transition probabilities between quantized states of the time series data. First, the time series data $x_t = \{x_1, x_2, \dots, x_n\}$ is normalized and then quantized into a finite number of states, typically corresponding to different ranges of values. The normalization

scales the values to fit between $[-1, 1]$:

$$\tilde{x}_t = \frac{2(x_t - \min(x_t))}{\max(x_t) - \min(x_t)} - 1 \quad (1)$$

After normalization, the time series is quantized into k discrete levels. This is done by mapping the normalized values to one of k bins:

$$\hat{x}_t = \left\lfloor k \times \frac{\tilde{x}_t - \min(\tilde{x}_t)}{\max(\tilde{x}_t) - \min(\tilde{x}_t)} \right\rfloor \quad (2)$$

Here, k is the number of quantization levels, and $\hat{x}_t$ is the quantized version of the time series. In the next step, we compute the transition probabilities. For each pair of quantized states (i, j) , the probability of transitioning from state i to state j is calculated based on the frequency of such transitions in the time series:

$$P_{ij} = \frac{\sum_{t=1}^{n-1} \mathbb{1}(\hat{x}_t = i \wedge \hat{x}_{t+1} = j)}{\sum_{t=1}^{n-1} \mathbb{1}(\hat{x}_t = i)} \quad (3)$$

where $\mathbb{1}$ is the indicator function that is 1 if the condition is true, and 0 otherwise. The MTF is then constructed as a $k \times k$ matrix where each element M_{ij} represents the transition probability P_{ij} as $M_{ij} = P_{ij}$. The resulting MTF matrix M can be visualized as an image, where each pixel intensity corresponds to the transition probability between states. This transformation effectively encodes the dynamical relationships and dependencies between different states of the time series into a 2D spatial format. This format is suitable for processing by CNNs for tasks such as classification, regression, or anomaly detection. By following these steps, the time series data is transformed into an image-like representation, making it possible to apply advanced image processing techniques for various analytical tasks.

It is important to note that to adjust the size of the k -sized windows of time-series data to either a smaller or larger size, several methods can be applied. One common technique is resampling, which involves either downsampling to reduce the size or upsampling to expand the size of the data windows. Therefore, to modify the output image dimensions, the quantization levels are initially adjusted to match the desired image dimension.

Figure 3 shows the result of transforming a time-series data sample using the reshaping and MTF method into an image-like matrix suitable for use by CNNs.

4.2 Dual-stage deep learning for HT detection and identification

The images obtained by transformation in previous sections then undergo processing through a series of CNN blocks in a two-path structure within the model's architecture, as

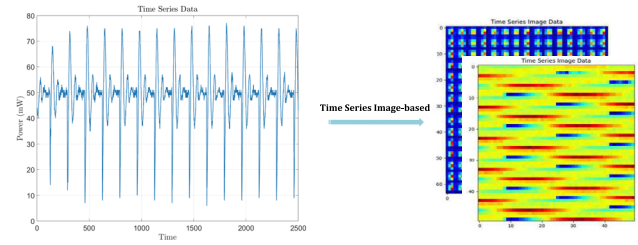


Fig. 3 Visualizing Time Series with MTF and Reshaping: (Left) Original Time Series, (Right) MTF and Reshaping Representation

presented in Figure 4. Each CNN block in the model applies a set of filters to the input image to extract features. This feature extraction is enhanced through batch normalization, ensuring stability and efficiency in training. Following the convolutional operation, the model employs a non-linear transformation, utilizing a ReLU function. This step introduces non-linearity to the model, allowing it to learn more complex patterns in the data. The transformed data is then passed through a pooling layer, which reduces the dimensions (downsizing the output) while retaining the most important features. This downsizing is crucial for reducing computational load and preventing overfitting by abstracting the features. The sequential arrangement of CNN and pooling layers is repeated across the model, ensuring thorough feature extraction and representation for accurate classification. The output from the model's convolutional layers in each path then proceeds through a flattening step, transforming the multi-dimensional feature maps into one-dimensional vectors, which are then concatenated. The concatenated output feeds into a series of densely connected (fully connected) layers. The final step of this process is a Softmax classifier, which assigns the predicted category, indicating whether an HT attack is triggered or not.

The feature extraction module, integrated with the classifier component, constitutes the model for the attack detector and identifier part as shown in Figure 5. The proposed model operates as a two-stage framework specifically designed to enhance HT attack detection and classification. In the first stage, named the Attack Detector, the model processes the transformed time-series data to determine whether it indicates a normal state or suggests the presence of a HT attack. Upon detecting an HT attack, the model activates the second stage, the Attack Identifier. This stage is tasked with classifying the specific type of Trojan attack identified initially. This two-stage approach not only ensures the accurate detection of potential security breaches but also enables a detailed classification of the specific type of threat, thereby facilitating more targeted and effective countermeasures. The model's architecture is designed to integrate these stages seamlessly, ensuring efficient and robust performance in the identification and categorization of hardware Trojan attacks.

Fig. 4 The feature extraction block using two 2D image representations from time series data

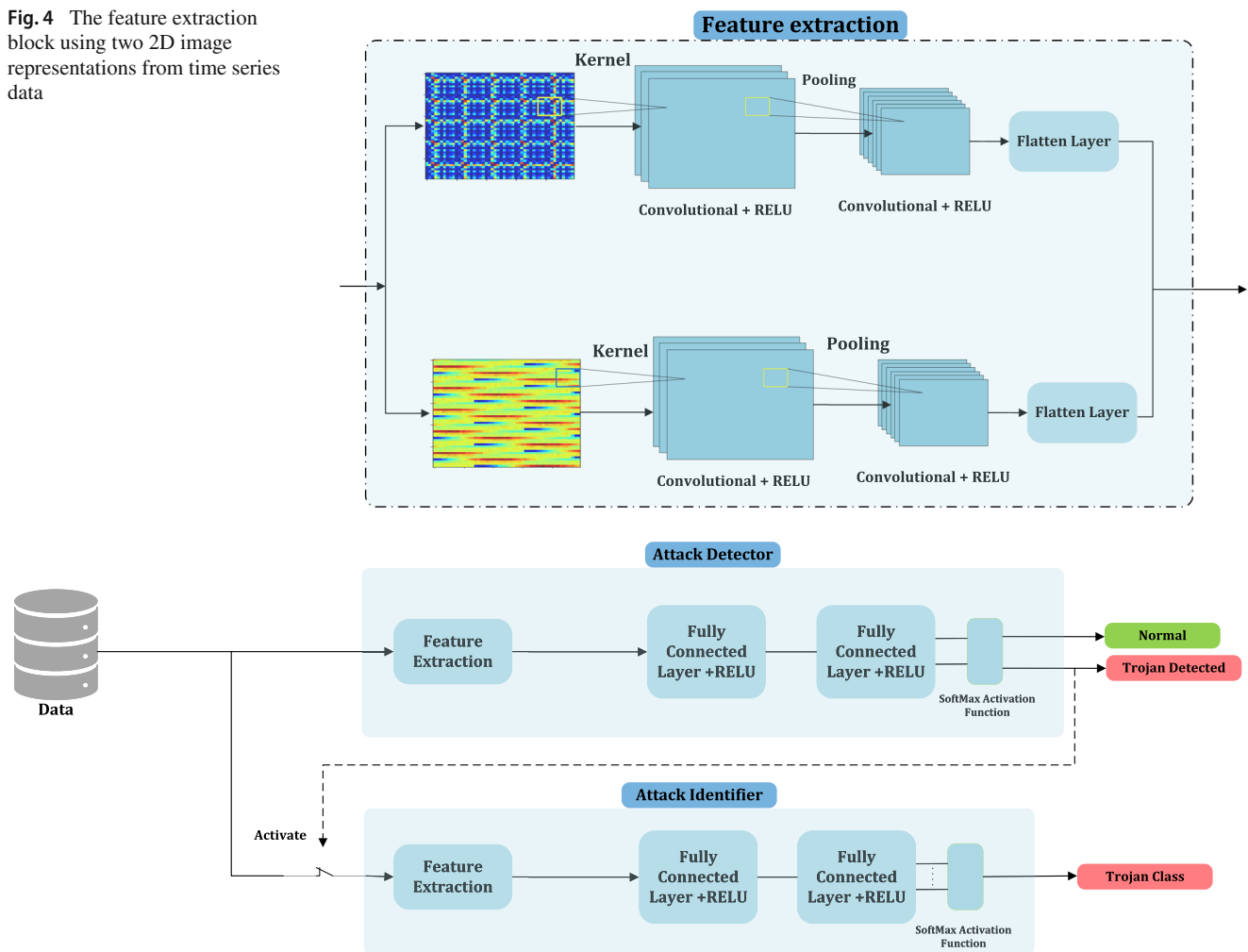


Fig. 5 Overview of the model for detection and identification of triggered Trojan attack based on the side-channel signal analysis

Remark 1 As shown in Figure 4, the proposed time-series-to-image transformations create a dual-path feature extraction framework. This approach ensures that different aspects of the data are captured and processed effectively. The reshaped time-series representation preserves the sequential structure of the signal, making it particularly effective at detecting abrupt changes, localized variations, and sharp transitions. Meanwhile, the MTF transformation encodes the temporal dependencies and cyclic behavior of the signal, allowing the model to recognize subtle state transitions and long-range correlations. By processing these representations through parallel CNN pathways, the network learns specialized feature sets from each domain. The reshaped representation enhances sensitivity to localized anomalies, while the MTF representation provides a broader temporal context. The extracted features from both branches are then concatenated, forming a richer and more discriminative feature space for HT detection. This synergistic feature extraction process improves classification accuracy and enhances robustness

against noise, process variations, and real-world perturbations.

It should be noted that the proposed dual-path CNN framework can enhance generalization by using multiple side-channel features, including power consumption and electromagnetic emissions. By combining MTF and reshaping techniques, the model effectively captures variations across different HT implementations, making it adaptable to diverse attack scenarios.

In the following section, the algorithm for training and testing the proposed model is explained.

4.3 Training and evaluation of the two-stage HT analysis model

As discussed, a robust two-stage deep learning framework is developed, specifically designed for enhancing HT attack detection and identifying distinct Trojan types. The framework consists of two key stages: the Attack Detector and the

Attack Identifier. The Attack Detector stage aims to determine whether the input data, in the form of side-channel signals, exhibits characteristics indicative of an HT attack or represents normal operation. If an attack is detected, the data is then passed to the Attack Identifier stage, which classifies the specific type of Trojan attack present in the data. To effectively train this two-stage framework, the input data, consisting of side-channel signals, is divided into two subsets: one containing signals associated with disabled HTs, and the other containing signals corresponding to triggered HTs. During training, the main aim is to minimize the error between the actual label of the input data (disabled or triggered HT) and the model's output.

The input data for Algorithm 1, partitioned into the two aforementioned subsets: D_{attack} and D_{normal} . This partition allows for targeted analysis where D_{attack} includes data representing various attack scenarios, and D_{normal} comprises data captured under normal operating conditions. Initially, each sample x_t from the dataset D undergoes a standard normalization process to ensure uniformity in the data scale. Subsequently, each normalized sample x_{norm} is transformed into a MTF, which provides a two-dimensional representation of the time-series data, capturing the transitional dynamics. Additionally, each sample is reshaped into a format suitable for CNN processing. The features extracted from both the MTF transformation and reshaping are concatenated to form a comprehensive feature set X_{feature} . A CNN is then trained on these features to classify each instance as either an 'attack' or 'normal' operation, based on the binary labels assigned.

For samples identified as attacks, a further distinction is made to classify the type of Trojan involved. To train this model, each attacking sample x'_t from D_{attack} undergoes a similar process of normalization, MTF transformation, and reshaping. The resulting features are concatenated into X'_{feature} , on which a second CNN is trained. This model categorizes the attack into predefined Trojan types, using the categorical labels that correspond to the specific types of Trojans. Both models are trained using features specifically engineered to capture both the general characteristics of attacks and the unique signatures of various Trojan types.

The output of the training phase consists of two trained models: one for detecting whether an attack has occurred and another for identifying the specific type of Trojan, if an attack is detected. By implementing this dual-model architecture, our approach not only improves the accuracy of attack detection but also enhances the accuracy of threat identification, which is critical for deploying timely and effective countermeasures in HT detection.

The testing phase, outlined in Algorithm 2, evaluates the trained model explained in Algorithm 1 by processing the sample through two models sequentially. Initially, the sample undergoes normalization to maintain consistency across the dataset, which is then transformed into a MTF and reshaped

Algorithm 1 Training Phase: Model Training for Attack Detection and Trojan Type Identification

```

1: Input: Time-series dataset  $D$  consisting of side channel signals, with  $D = D_{\text{attack}} \cup D_{\text{normal}}$ . Here,  $D_{\text{attack}}$  contains signals indicating attacks and  $D_{\text{normal}}$  contains signals from normal operations. Each sample in  $D$  has dimensions  $n \times m$ , where  $n$  is the number of samples and  $m$  is the length of each time-series.
2: Model 1: Attack Detection
3: Data Normalization and Transformation:
4: for each sample  $x_t \in D$  do
5:    $x_{\text{norm}} \leftarrow \text{Norm}(x_t)$ 
6:    $X_{\text{MTF}} \leftarrow \text{MTF}(x_{\text{norm}})$ 
7:    $X_{\text{reshaped}} \leftarrow \text{Reshape}(x_{\text{norm}})$ 
8:    $X_{\text{feature}} \leftarrow \text{Concat}(X_{\text{MTF}}, X_{\text{reshaped}})$ 
9: end for
10: Labels: Binary (attack/no attack)
11: Train a CNN using  $X_{\text{feature}}$  to differentiate between normal operation and attack scenarios.
12: Model 2: Trojan Type Identification
13: Input: Samples  $x'_t$  taken from  $D_{\text{attack}}$ , where signals indicate an attack.
14: Data Normalization and Transformation:
15: for each sample  $x'_t \in D_{\text{attack}}$  do
16:    $x'_{\text{norm}} \leftarrow \text{Norm}(x'_t)$ 
17:    $X'_{\text{MTF}} \leftarrow \text{MTF}(x'_{\text{norm}})$ 
18:    $X'_{\text{reshaped}} \leftarrow \text{Reshape}(x'_{\text{norm}})$ 
19:    $X'_{\text{feature}} \leftarrow \text{Concat}(X'_{\text{MTF}}, X'_{\text{reshaped}})$ 
20: end for
21: Labels: Categorical, based on the number of Trojan types.
22: Train a separate CNN using  $X'_{\text{feature}}$  to classify the type of Trojan.
23: Output: Two trained models - one for detecting attacks and another for identifying Trojan types.

```

for compatibility with CNN architectures. These transformations facilitate comprehensive feature extraction, represented by X_{feature} , which is input into the first CNN model to determine the presence of an attack. If an attack is detected, the same sample is reprocessed using the second model, which further analyzes the normalized and transformed data to classify the specific Trojan type. This dual-stage approach ensures both detection and detailed classification of potential security threats within the system.

Algorithm 2 Testing Phase: Model Testing for Attack Detection and Trojan Type Identification

```

1: Input: A single time-series sample  $x_t$  consisting of power traces with dimensions  $m$ , where  $m$  is the length of the time-series.
2: Model 1: Attack Detection
3: Data Processing:
4:  $x_{\text{norm}} \leftarrow \text{Norm}(x_t)$ 
5:  $X_{\text{MTF}} \leftarrow \text{MTF}(x_{\text{norm}})$ 
6:  $X_{\text{reshaped}} \leftarrow \text{Reshape}(x_{\text{norm}})$ 
7:  $X_{\text{feature}} \leftarrow \text{Concat}(X_{\text{MTF}}, X_{\text{reshaped}})$ 
8: Prediction  $\leftarrow$  Predict using trained Attack Detector with  $X_{\text{feature}}$ 
9: if Prediction is "attack" then
10:   Activate Attack Identifier model: Trojan Type Identification
11:   Trojan Type  $\leftarrow$  Predict using trained Attack Identifier with  $X_{\text{feature}}$ 
12: end if
13: Output: Detection result from Attack Detector model (attack/no attack) and if attack, identification the time-series signal from Attack identifier (Trojan type).

```

4.4 Monte carlo dropout for uncertainty-aware HT detection

Accurately identifying malicious modifications in hardware designs is crucial, but equally important is quantifying the uncertainty in these detections to enhance reliabil-

ity and trustworthiness. Uncertainty modeling provides a more detailed understanding of detection outcomes, enabling informed decision-making. In this work, MCD is used to improve the model's predictive capability in HT detection by not only identifying malicious modifications but also quantifying the confidence in these detections.

Dropout is a regularization technique used during training to prevent overfitting by randomly setting a fraction of the input units to zero at each update. In Bayesian inference, the goal is to find the posterior distribution $p(\mathbf{W} \mid \mathcal{D})$ over the model parameters $\mathbf{W}$ given the data $\mathcal{D}$. The posterior can be intractable to compute, but by using dropout, one can approximate it with a variational distribution $q(\mathbf{W})$. In fact, variational inference is used to approximate the true posterior distribution $p(\mathbf{W} \mid \mathcal{D})$ with a simpler distribution $q(\mathbf{W})$. The goal is to minimize the Kullback-Leibler (KL) divergence between $q(\mathbf{W})$ and $p(\mathbf{W} \mid \mathcal{D})$:

$$D_{KL}(q(\mathbf{W}) \parallel p(\mathbf{W} \mid \mathcal{D})) \quad (4)$$

In the context of MCD, the variational distribution $q(\mathbf{W})$ is defined over matrices whose elements are randomly set to zero according to a Bernoulli distribution. Specifically:

$$\mathbf{W}_i = \mathbf{R}_i \cdot \text{diag} \left([\mathbf{z}_{i,j}]_{j=1}^{\kappa_i} \right), \quad (5)$$

where $\mathbf{z}_{i,j} \sim \text{Bernoulli}(p_i)$ for $i = 1, \dots, L$, $j = 1, \dots, \kappa_{i-1}$ and p_i represents predefined probabilities, and $\mathbf{R}_i$ are matrices serving as variational parameters. In this context, the binary variable $\mathbf{z}_{i,j} = 0$ indicates that unit j in layer $i - 1$ is dropped as an input to layer i . Therefore, the predictive distribution of output for a sample input $\mathbf{x}$ can be approximated as:

$$p(\mathbf{y} \mid \mathbf{x}, \mathcal{D}) \approx \int p(\mathbf{y} \mid \mathbf{x}, \mathbf{W}) q(\mathbf{W}) d\mathbf{W} \quad (6)$$

Applying dropout during inference enables sampling from this approximate posterior distribution to compute the predictive mean as follows:

$$\hat{\mathbf{y}} = \frac{1}{N} \sum_{i=1}^N f(\mathbf{x}, \mathbf{W}_1^i, \dots, \mathbf{W}_L^i) \quad (7)$$

Here, $f(\mathbf{x}, \mathbf{W}_1^i, \dots, \mathbf{W}_L^i)$ denotes the output of the network for the i -th sample of the weights. The mean of these stochastic forward pass predictions provides the final prediction, while the variance estimates the epistemic uncertainty:

$$\text{Var}(\hat{\mathbf{y}}) = \frac{1}{N} \sum_{i=1}^N (\hat{\mathbf{y}}_i - \hat{\mathbf{y}})^2 \quad (8)$$

where $\hat{\mathbf{y}}_i = f(\mathbf{x}, \mathbf{W}_1^i, \dots, \mathbf{W}_L^i)$.

Remark 2 MCD approximates Bayesian inference by using dropout during both training and inference to sample from an approximate posterior distribution over network weights. This approach provides a practical way to estimate uncertainties, enhancing the reliability and robustness of the HT detection mechanism in real-world applications by allowing for better decision-making based on uncertainty quantification. Unlike deterministic CNN classifiers that may overfit to specific patterns, MCD introduces an uncertainty-aware mechanism that reduces misclassification rates and dynamically adjusts detection thresholds. This probabilistic modeling approach allows our framework to adapt to varying confidence levels, ensuring more reliable Trojan detection across diverse datasets. High variance in predictions indicates low confidence in detection, suggesting that the HT detector may be less reliable under those conditions. By flagging such cases, MCD mitigates overconfidence in uncertain predictions and improves classification accuracy, particularly under noisy environments. However, it is important to consider the computational cost associated with performing multiple forward passes during inference.

It should be noted that MCD enhances the reliability of HT detection by quantifying uncertainty, ensuring that the model does not make overconfident predictions in ambiguous cases. The decision-making process is based on the variance of predictions across multiple stochastic forward passes. Higher variance indicates low confidence, prompting the model to flag such cases for further analysis. To optimize classification reliability, a probability threshold is applied alongside the standard deviation threshold to determine whether a prediction should be accepted or rejected. The selection of these thresholds is guided by experimental tuning, ensuring a balance between detection accuracy and false positive minimization, which is further analyzed in the results section.

4.5 Overview of Dataset

The dataset employed for this research was obtained from the publicly available IEEE DataPort website, as described in the work by [58, 59]. This dataset comprises power side-channel signals captured from a subset of HTs referenced in the TrustHub repository, which were specifically designed to target a cryptographic core circuit implementing the Advanced Encryption Standard (AES) algorithm with a 128-bit key length. The AES algorithm is a widely used cryptographic technique for securing sensitive data through encryption.

The AES circuit receives a 128-bit plaintext input and encrypts it using a secret key to produce a 128-bit ciphertext output. For data collection, the encryption process is executed 10,000 times, generating an equal number of time-series signals with a size of 2500. As described in [59], the

data is collected using two Sakura-G boards, focusing on an AES circuit to capture power consumption that indicates the presence or activation of HTs. This dataset contains data gathered under various conditions, including when the HT is disabled or actively triggered. The dataset's uniqueness lies in its coverage of both static and dynamic components of power consumption across different operational states of HTs. This provides a rich basis for developing and evaluating HT detection models. This detailed dataset enables the development of a statistical model aimed at improving the accuracy and robustness of HT detection methods.

It is important to note that the ideal power consumption of an AES circuit within a golden chip consists of two parts of dynamic power and static power [60]:

$$P_T = P_D + P_S \quad (9)$$

where, P_T represents the overall intrinsic power consumption, comprising both the dynamic power, denoted by P_D , and the static power, represented by P_S . However, power measurements can be affected by noise due to process variations, supply voltage fluctuations, temperature changes, and measurement limitations [61]. The observed power consumption is thus given by:

$$\tilde{P}_T = P_T + P_N \quad (10)$$

where P_N is noise power. After activation of a HT, additional power consumption is introduced to the circuit. This extra consumption changes the observed power data and can be expressed as:

$$\mathcal{P}_T = \tilde{P}_T + P_{TR} \quad (11)$$

where P_{TR} represents the power added by the triggered Trojan. By analyzing and comparing the power signals $\mathcal{P}_T$ and $\tilde{P}_T$, it is feasible to detect the presence of an active HT compared to its dormant state. However, one of the significant challenges in side-channel-based HT detection is the presence of noise. The HT power consumption signature can easily be obscured by noise. This problem of noise can mask the additional power consumed by an HT, greatly reducing the accuracy of side-channel-based attack detection methods. Consequently, distinguishing between the normal power consumption of an AES circuit and the altered consumption due to an HT becomes a complex task.

In this paper, we focus on five specific Trojan types for the sake of comparison with previous works, out of the twelve documented in the available dataset. These are namely AES-T500, AES-T600, AES-T700, AES-T800, and AES-T1600. These benchmarks describe various scenarios in which a HT can be triggered and the subsequent impacts on the AES circuit [59]. The AES-T500 is activated by a specific sequence

of numbers and then drains the battery through a perpetually rotating shift register. The AES-T600 triggers upon receiving a predefined number and then leaks the secret key by incrementing the leakage current for each 0 bit within the key. Similarly, both the AES-T700 and AES-T800 are activated by a certain number and sequence of numbers, respectively, and both leak the key through a covert channel using a Code Division Multiple Access (CDMA) sequence code. Lastly, the AES-T1600, which also responds to a specific sequence of numbers, leaks the key via an RF signal, potentially allowing remote eavesdropping. Each benchmark presents a unique combination of trigger conditions and payloads, demonstrating the diverse and insidious nature of HT attacks on AES.

From the dataset, which includes 10,000 time-series signals with the disabled Trojan and another 10,000 with the triggered Trojan, a selection is made where 80% of the signals from each set are allocated for the training set and 20% for the validation set, ensuring robust training and validation for HT detection in AES circuits.

5 Results and discussions

In this section, results and discussion are provided about the proposed HT detection method. First, the performance of the proposed HT detection method is evaluated using the dataset discussed earlier. Then, the robustness of our method is assessed against noise in the validation dataset, a critical factor for evaluating the approach method's real-world applicability. Finally, the impact of noise on discriminability between disabled and triggered Trojan signals is discussed. This is essential for assessing the effectiveness of the proposed HT detection method.

For the purpose of simulation and evaluation of our proposed time series imaging-based learning method, a selection is made where 80% signals are allocated for the training set and 20% for the validation set. In our study, we explore the application of the model, as depicted in Figure 5 and implemented using PyTorch version 2.1.2, to accurately detect HT attacks.

The model's initial analysis begins with two convolutional layers dedicated to feature extraction. The first layer employs 32 filters, each with a kernel size of 5, a stride of 1, and padding of 2. This configuration allows the model to capture the initial feature set from input. A subsequent max pooling layer, with a kernel size of 2 and a stride of 2, reduces the spatial dimensions of the feature maps, retaining only the most significant information. To deepen the analysis, the second convolutional layer utilizes 64 filters, maintaining the same kernel size, stride, and padding to further refine the extracted features. Following feature extraction, the model proceeds to the classification phase with dedicated fully con-

nected layers for each stage. For the Attack Detector, the first fully connected layer maps the concatenated layer into a 128-dimensional space, and the subsequent layer condenses this data into a 2D output, aligning with the classification targets. Moreover, the Attack Identifier uses the first fully connected layer to map features into a 256-dimensional space, followed by a second layer that produces a 5-dimensional output for more detailed classification of potential threats. Table 1 presents the CNN model architecture parameters.

In this study, we employ the cross-entropy loss function for both binary and multi-class classification tasks. The binary classification loss is expressed as follows:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)] \quad (12)$$

where y represents the true label, $\hat{y}$ the predicted probability of the positive class, and N the number of samples in the dataset. Here, y_i indicates the presence or absence of an HT, and $\hat{y}_i$ is the predicted probability that the i -th sample contains an HT.

For the Attack Identifier, which involves multi-class classification, the loss function is adapted to cater to multiple classes and is defined as:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C [y_{i,c} \cdot \log(\hat{y}_{i,c})] \quad (13)$$

where C is the number of classes, $y_{i,c}$ is a binary indicator of whether class label c is the correct classification for observation i , and $\hat{y}_{i,c}$ is the predicted probability of observation i being of class c .

Accuracy for both binary and multi-class scenarios is calculated by:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\hat{y}_i = y_i) \quad (14)$$

where $\mathbb{1}(\cdot)$ is the indicator function that is 1 when $\hat{y}_i$ equals y_i and 0 otherwise.

5.1 Comparative analysis

To assess the effectiveness of our proposed methods for HT detection, we conducted an extensive evaluation, comparing our approach to a range of established models. This evaluation includes the One-Class Support Vector Machine (OC-SVM), renowned for its anomaly detection efficacy. We also explored three distinct neural network architectures: MLP, CNN, and Hierarchical Temporal Memory (HTM)

networks, the latter modeled after the human neocortex to handle temporal data processing. The performance metrics for these models are sourced from the results documented in [59]. Moreover, our comparison extends to the innovative Attention-Res-Attention (ARA) network, as detailed in [9]. This method combines the ResNeXt network structure with an attention mechanism, creating the ARA network designed to specifically enhance the detection of critical features indicative of HTs.

5.1.1 Comparative performance evaluation of the attack detector model

To establish a comprehensive comparative analysis, we first trained and evaluated the proposed Attack Detector model, depicted in Figure 5, on each of the AES datasets. The results of this evaluation are presented in Table 2, which provides a detailed comparison of the detection rates achieved by various models. As evidenced by the table, our proposed time-series imaging-based detection approach exhibits superior performance on the AES-600 and AES-1600 hardware Trojan benchmarks, while maintaining comparable performance to existing methods on the AES-500, AES-700, and AES-800 benchmarks.

The proposed model achieves an accuracy of approximately 95.25% on the AES-600 benchmark and 77.50% on the AES-T1600 benchmark, both exceeding the accuracy levels reported in [9, 59]. These results underscore the efficacy of our approach in accurately detecting hardware Trojan attacks, particularly in scenarios where existing methods may not be effective. The comprehensive analysis presented in Table 2 substantiates the competitive performance of our proposed methodology, demonstrating its potential to advance the state-of-the-art in hardware Trojan detection and identification.

The AES-T1600 dataset is particularly challenging compared to other datasets for HT detection due to its complex signal patterns and subtle variations when the Trojan is activated as shown in Figure 6. Despite these difficulties, our proposed method achieves an accuracy of 77.50%, which is a notable accuracy compared to the reported accuracy of 59.6% in [59] and 63.1% in [9]. This result demonstrates the effectiveness of the approach in addressing HT detection difficulties in challenging datasets.

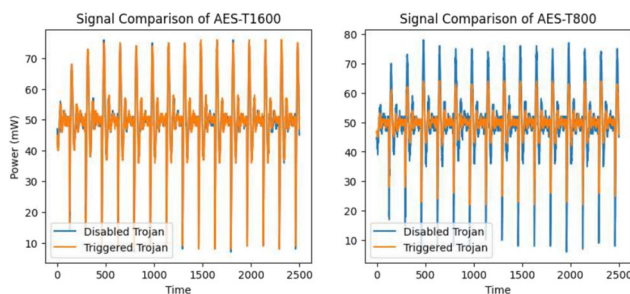
Table 3 shows the Recall, Precision, and F1-Score metrics for the model across five different AES benchmarks. As evidenced by the data, the model exhibits robust performance with high scores in Precision and Recall for the AES-T700 and AES-T800 benchmarks, leading to an F1-Score of 1.00 in both cases. These results indicate that the model is highly effective at accurately detecting hardware Trojans without any false negatives or false positives in these datasets. However, in more challenging datasets such as AES-T600 and

Table 1 CNN Model Architecture for Attack Detector and Attack Identifier

Layer Type	Filters/Nodes	Kernel Size	Stride	Padding
Convolutional Layer 1	32	5	1	2
Batch Normalization Layer 1	32	-	-	-
Max Pooling Layer	-	2	2	-
Convolutional Layer 2	64	5	1	2
Batch Normalization Layer 2	64	-	-	-
Max Pooling Layer	-	2	2	-
Fully Connected Layer 1 (Attack Detector)	128	-	-	-
Fully Connected Layer 2 (Attack Detector)	2	-	-	-
Fully Connected Layer 1 (Attack Identifier)	128	-	-	-
Fully Connected Layer 2 (Attack Identifier)	5	-	-	-

Table 2 Comparison of accuracy of Attack Detector with different methods on each AES Datasets

	AES-T500	AES-T600	AES-T700	AES-T800	AES-T1600
CNN	84.5%	55.5%	100%	100%	64%
MLP	84%	65.5%	100%	100%	70.5%
OC-SVM	89.7%	49.7%	90.2%	92.7%	50.2%
HTM [59]	100%	63.5%	98.1%	100%	59.6%
ARA [9]	100%	67.3%	100%	100%	63.1%
Our Method	98.5%	95.25%	100%	100%	77.50%

**Fig. 6** Signal comparison of AES-T1600 (left) and AES-T800 (right)

particularly AES-T1600, while the model still demonstrates strong Recall and Precision, the F1-Scores slightly decrease to 0.9467 and 0.7737, respectively.

The performance of the proposed model for detecting HTs in AES encryption modules for the AES-600 and AES-1600 HT benchmarks is demonstrated in Figures 7 and 8, which show the model's training and validation loss as well as the accuracy of the model on these dataset.

5.1.2 Comparative performance evaluation of the attack identifier model

In this scenario, we investigate a detailed comparative performance analysis of our Attack Identifier model, specifically focusing on its capacity to identify various HT types across different AES benchmarks. Table 4 presents a comparative analysis of the accuracy of Trojan identification between

our method and the ARA network as reported by [9]. The proposed Attack Identifier model demonstrates exceptional identification accuracy across all benchmarks, notably achieving perfect scores of 100% in the AES-T600, AES-T800, and AES-T1600 datasets. This outperforms the ARA network, where our model's average accuracy reaches 99.8%, surpassing the 97% average accuracy reported for the ARA network [9]. This comparative analysis underscores the efficacy of our approach to accurately detecting hardware Trojan attacks.

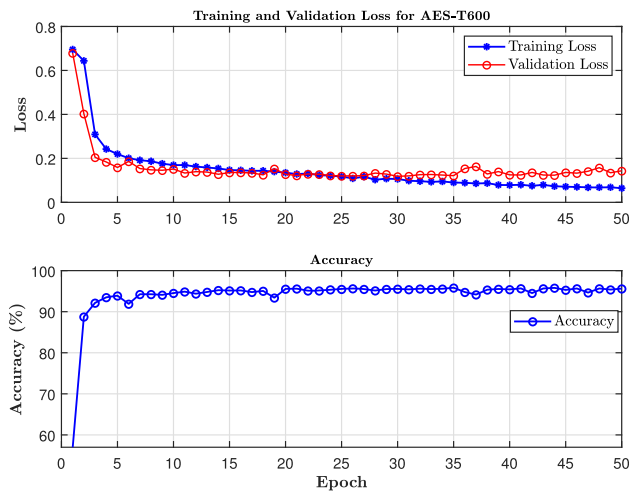
The plot in Figure 9 illustrates the training, validation loss and overall accuracy over the epochs.

5.1.3 Comparative performance evaluation of the attack detector and identifier model

In this section, we evaluate the effectiveness of dual-path for feature extraction for the proposed dual-stage Attack Detector and Attack Identifier model, as illustrated in Figure 5. This model is specifically designed to enhance the accuracy of detecting and classifying HT attacks in two distinct stages. For the Attack Detector, the model is trained using the dataset benchmark, which encompasses both triggered HT scenarios and normal operating conditions. Specifically, the training dataset includes 80% of all data evenly split between Trojan-triggered and Trojan-disabled samples to ensure a comprehensive learning experience. This approach allows the model to learn distinguishing features of both attack and

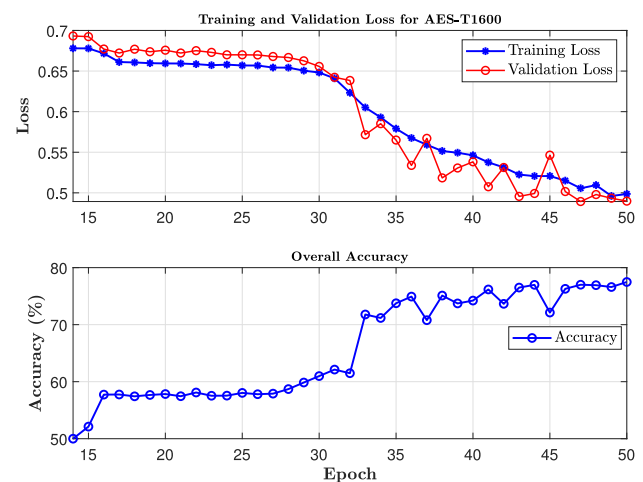
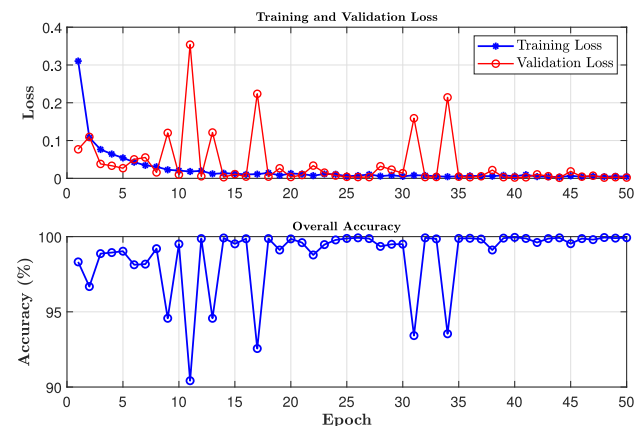
Table 3 Performance Metrics of the Attack Detection Model across AES Benchmarks

	AES-T500	AES-T600	AES-T700	AES-T800	AES-T1600
Recall	0.9855	0.9525	1	1	0.775
Precision	0.9857	0.9527	1	1	0.78
F1-Score	0.9855	0.9525	1	1	0.7737

**Fig. 7** Validation loss and accuracy for AES-600

non-attack states effectively. The Attack Identifier, which is built upon the Attack Detector's model, utilizes the trained Attack Detector as its preliminary filtering stage. This staged approach refines the focus of the Attack Identifier, which then classifies the type of Trojan attack more accurately.

It should be noted that an alternative model is to classify the normal and all attack types in a one-stage model as shown in Figure in 10. This comparative model simultaneously classifies both normal and Trojan data without the intermediary step of a dedicated Attack Detector. However, the two-stage model with binary classification in the first layer offers simplicity and efficiency. In fact, for initial anomaly detection, the Attack Detector stage in the two-stage model can quickly identify whether the data is normal or an attack, serving as an early warning system that triggers the Attack Identifier for more detailed analysis if needed. Moreover, fine-tuning this Attack Detector model for new types of attack can be more effective. In scenarios where different attack types emerge, fine-tuning allows the model to adapt without retraining from scratch, saving time and computational resources. For a one-stage model, however, adding new types of attacks requires

**Fig. 8** Validation loss and accuracy for AES-1600**Fig. 9** Training and validation loss curves, and overall accuracy progression during the training of the Attack Identifier model

fine-tuning and adding additional classes to the model, which increases complexity and resource demands. Here, the performance of this dual-stage model is compared against the one-stage model, as depicted in Figure 10. The dual-stage model achieves an overall accuracy of 93.23%, compared

Table 4 Comparison of the accuracy of Trojan identification

	AES-T500	AES-T600	AES-T700	AES-T800	AES-T1600	Average Accuracy
ARA [9]	100%	91.1%	97.8%	100%	97.8%	97%
Attack Identifier	99.7%	100%	99.8%	100%	100%	99.8%

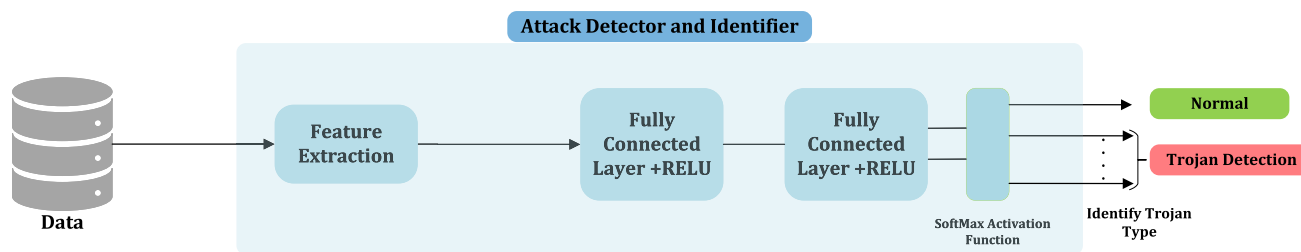


Fig. 10 One stage Attack Detector and Identifier

Table 5 Comparison of the accuracy of Trojan Detection for one-stage vs two-stage model

	AES-T500	AES-T600	AES-T700	AES-T800	AES-T1600	Normal Accuracy
One-Stage	95.15%	93.2%	100%	99.15%	75.4%	92.5%
Two-Stage	96.45%	93.85%	100%	100%	77.5%	92.9%

to the single-stage model's accuracy of 92.54% as shown in Table 5.

The one-stage model differentiates between attack types. This model provides detailed information about the specific type of attack, which is valuable for targeted responses and mitigation strategies, but it is more complex and may take longer to train. On the other hand the two-stage model, which first uses a binary classifier to detect whether the data is normal or an attack. If an attack is detected, the data is then passed to a multi-class classifier that identifies the specific type of attack. This hierarchical approach breaks down the problem into simpler sub-problems, allowing each classifier to specialize in its task. This method can be more efficient, as the second stage classifier only runs when an attack is detected, potentially reducing computational load.

5.2 Analyzing the performance of the attack detector model using MCD

To improve the detection capability of the attack detector for AES-T600 and AES-T1600 datasets, MCD techniques can be employed. To this aim, in the detection process, the model used the standard deviation of predictions generated by MCD to determine whether to accept or reject a prediction. Specifically, the maximum predicted probability and the corresponding uncertainty (standard deviation) are compared against threshold. Predictions with high uncertainty, indicating low confidence, are either rejected or flagged for further analysis. This approach ensured that only confident detections are considered, reducing the risk of erroneous classifications.

The top Figures 12 and 13 show the validation accuracy for different combinations of standard deviation and probability thresholds. It illustrates that reducing the standard deviation threshold (moving upwards in the plot) consistently improves accuracy. This indicates that focusing on more con-

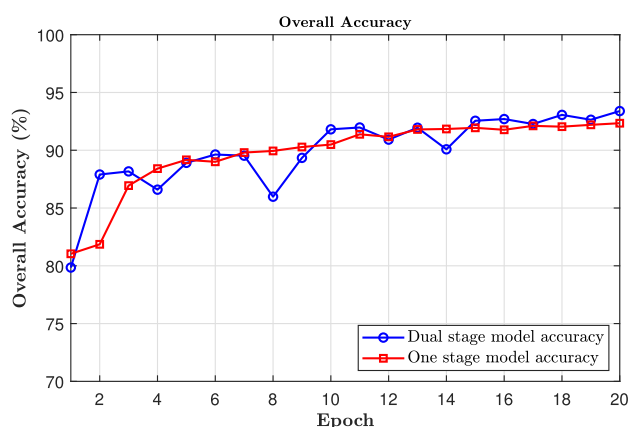


Fig. 11 Comparison of overall accuracy of dual stage and one stage model

fidant predictions (with lower uncertainty) results in higher accuracy. In contrast, increasing the probability threshold alone has a less significant effect on improving accuracy. This demonstrates that modeling uncertainty through MCD is more effective than relying solely on higher probability thresholds. The bottom Figures in 12 and 13 display the acceptance rate, showing how many predictions are accepted under different threshold combinations. A higher acceptance rate is achieved with larger standard deviation thresholds (moving downward in the plot), but this comes at the cost of reduced accuracy, as seen in the top figure. This highlights the trade-off between accepting more predictions and maintaining high accuracy, with MCD providing the ability to adjust this balance based on uncertainty. Therefore, the optimal probability and standard deviation thresholds are determined experimentally by evaluating their impact on detection accuracy and acceptance rates, as shown in Figures 12 and 13. The results indicate that reducing the standard deviation threshold improves accuracy by filtering out uncertain detections,

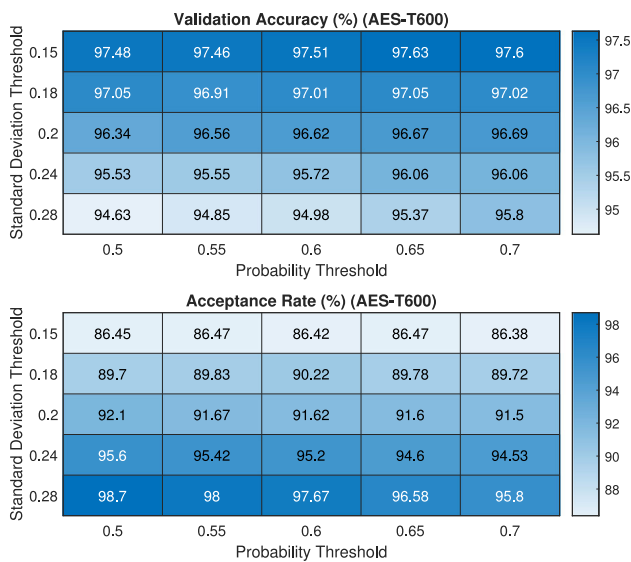


Fig. 12 Validation Accuracy and Acceptance Rate for AES-T600 with varying Probability and Standard Deviation Thresholds using MCD

while adjusting the probability threshold ensures that only high-confidence classifications are retained.

MCD introduces uncertainty quantification into the detection process, allowing the model to distinguish between high- and low-confidence predictions. A comparison of predictions with and without MCD demonstrates that variance-based thresholding effectively reduces misclassifications, ensuring that only reliable detections are accepted. The observed trade-off between accuracy and acceptance rate highlights the importance of probability and standard deviation thresholds in refining decision-making, particularly in security-critical applications where false positives must be minimized.

5.3 Robustness evaluation of the proposed HT detection method under noise

Evaluating the robustness of the proposed HT detection method using time series imaging in the presence of noise is essential, as hardware systems function in real-world environments that are subject to various disturbances. Factors such as process variations, aging, and voltage level fluctuations significantly impact power consumption patterns, potentially masking the presence of HTs or mimicking their effects. To address this, introducing white noise, represented as $\mathcal{N}(0, \sigma^2)$, into the test signals can effectively simulate these real-world conditions. This approach helps assess HT detection methods' resilience with and without using MCD technique. The model must also be tested across a range of noise levels to determine the threshold at which accuracy degrades. Comprehensive testing not only validates the robustness of the HT detection method, but also provides insights into the model's reliability and operational limits

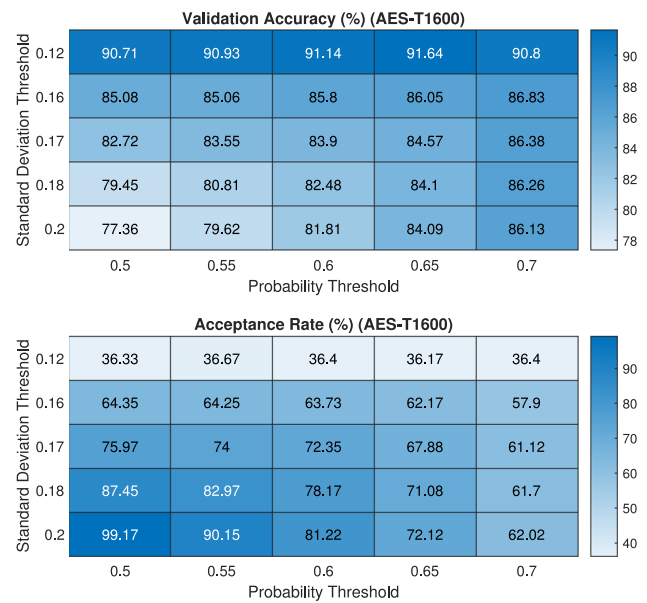


Fig. 13 Validation Accuracy and Acceptance Rate for AES-T1600 with varying Probability and Standard Deviation Thresholds using MCD

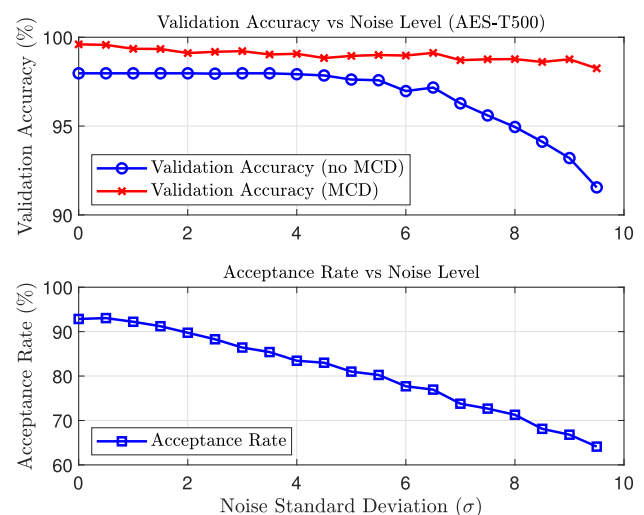


Fig. 14 Impact of noise on validation accuracy and acceptance rate with and without MCD method for AES-T500

in real-world situations [59]. The consistent detection accuracy under different noise conditions further demonstrates the model's ability to generalize across various HT variants and real-world environmental variations.

Figures 14 and 15 show how the validation accuracy of the proposed HT detection methods responds to varying noise levels within the collected signals of the HT benchmarks. These figures demonstrate the impact of noise on the validation accuracy of the model with and without MCD for AES-T500 and AES-T700, respectively. In the top plots, as the noise increases, the validation accuracy without MCD (blue line) decreases, whereas the validation accuracy with

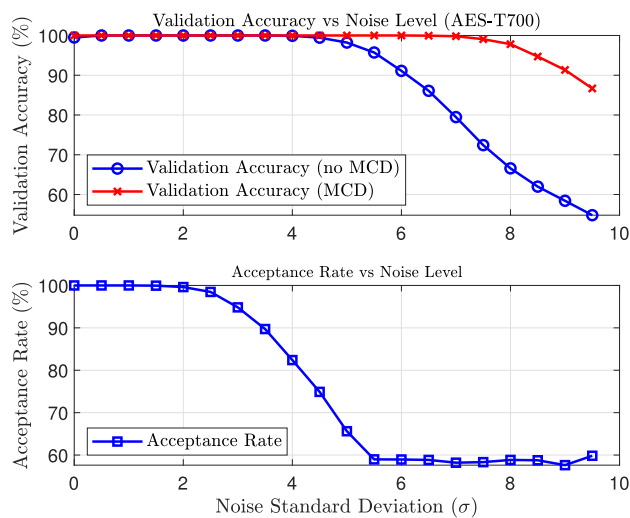


Fig. 15 Impact of noise on validation accuracy and acceptance rate with and without MCD method for AES-T700

MCD (red line) remains much more stable. It is important to note that although the proposed HT detector is somewhat robust to moderate noise levels, using MCD makes it significantly more resilient. This highlights the robustness of the MCD approach in mitigating the impact of noise. The bottom graphs show that the acceptance rate declines as noise levels increase. This indicates that the model becomes more cautious in accepting predictions under higher noise conditions, which aligns with the goal of reducing erroneous classifications when faced with uncertainty.

The application of MCD provides robustness against noise by generating multiple predictions and assessing their consistency. The MCD approach enables the model to maintain high detection accuracy even under noisy conditions, which is crucial for ensuring reliable hardware security.

5.4 Depth of results analysis and discussion

To provide a more comprehensive understanding of the proposed model's capabilities and limitations, this section provides a deeper analysis of the performance results presented earlier. We examine the differences in detection accuracy across various HT benchmarks, analyze the factors that contribute to these variations, and discuss the broader implications for generalization and practical deployment. This in-depth analysis helps explain the strengths of the proposed dual-stage framework while also highlighting challenges that must be addressed for real-world application.

5.4.1 Generalization and robustness discussion

This subsection discusses the model's ability to generalize across diverse Trojan variants and its robustness to real-

world variations, such as environmental noise and hardware uncertainty. In this study, real-world uncertainty is simulated by introducing Gaussian noise into the side-channel signals to represent environmental and manufacturing variations such as process variation, aging, and voltage drift. These perturbations cause data distribution shifts similar to those encountered when models are deployed in unseen settings. The proposed framework maintains high classification accuracy under these conditions, and the MCD mechanism further enhances generalization by flagging uncertain predictions for further analysis rather than forcing potentially incorrect decisions.

Moreover, the two-stage architecture-comprising an Attack Detector and a Trojan Identifier-contributes to generalization by dividing the detection and classification tasks. This separation reduces model complexity per task and allows for more targeted learning. It also makes the system easier to adapt to new Trojan types in future deployments, as the Trojan Identifier can be fine-tuned or extended without modifying the Attack Detector. Together, these design choices demonstrate the method's strong generalization capability, even in the absence of external datasets.

5.4.2 Variant-specific performance

Understanding how the proposed method performs across different Trojan variants is essential for identifying its strengths and limitations. This subsection analyzes detection trends in relation to the specific activation mechanisms and leakage characteristics of each HT benchmark.

The hardware Trojans AES-T600 and AES-T1600 are considered stealthy not only due to their minimal logic footprint, but also because of their subtle impact on power consumption and side-channel emissions [62]. AES-T600 operates by modulating the leakage current based on the secret key bits, where only the '0' bits increase power leakage marginally. This modulation is statistically minor and designed to blend into the normal power profile of the AES circuit, making it extremely difficult to isolate using conventional side-channel analysis like Differential Power Analysis (DPA). Its activation under rare input patterns further reduces the chance of being exposed during profiling or runtime monitoring. AES-T1600 is even more covert as it is shown in Figure 6. It leaks the key via RF emissions by toggling an unused I/O pin, bypassing standard power and logic-based detection entirely. The Trojan avoids affecting the main data or control paths and has a negligible effect on the aggregate power signature of the system. Since both Trojans either blend into or circumvent traditional side-channel vectors, they pose a significant challenge to detection methods that rely on identifying anomalies in power, timing, or electromagnetic signatures.

In contrast, AES-T700 and AES-T800 are relatively easier to detect due to their more pronounced and structured impact on side-channel emissions [59]. Both Trojans are designed to leak the secret key using a CDMA modulation scheme, which introduces repetitive, high-frequency activity in the circuit. This activity is embedded within the power consumption traces and often leads to detectable patterns, especially after transformation into image representations via MTF or reshaping. Furthermore, the activation of these Trojans is tied to specific and more frequent input conditions compared to AES-T600 or AES-T1600, which increases the likelihood of their presence being reflected in the training data. As a result, the proposed model, which uses spatial feature extraction from side-channel-derived images, is particularly effective in capturing the structured leakage patterns associated with these Trojan types.

AES-T500 represents an intermediate case in terms of stealth, being less covert than AES-T600 and AES-T1600, yet exhibiting less prominent side-channel signatures than AES-T700 and AES-T800. This Trojan drains the battery using a perpetually rotating shift register once a rare input sequence is observed. Although its activation leads to a sustained change in power consumption, the shift register logic introduces only a gradual increase in dynamic power, which may not create sharp or localized features in the side-channel trace. However, unlike RF-based or key-dependent modulation Trojans, AES-T500 still influences the core power signature, making it more amenable to detection through temporal analysis and spatial encoding. Accordingly, the proposed method achieves high accuracy in detecting AES-T500, though with slightly lower precision than CDMA-based Trojans.

This variant-specific challenge is reflected in our evaluation. The model achieved F1-scores of 1.0 for AES-T700 and AES-T800 presented in Table 3, whereas the score dropped to 0.77 for AES-T1600. This performance gap highlights the difficulty of detecting Trojans that produce minimal side-channel perturbations. Notably, when MCD is activated, the accuracy on AES-T1600 increased from 77.5% to over 91% at optimal uncertainty thresholds (Figure 13), confirming that MCD improves decision reliability in low-confidence scenarios.

5.4.3 Challenges in practical applications

Despite the strong performance of the proposed framework in controlled evaluations, practical deployment scenarios may introduce additional complexities that warrant consideration. One important factor is the fidelity of side-channel measurements, which in real-world conditions can be affected by environmental noise, temperature variation, PCB layout differences, and interference from concurrent processes. These sources of variation may distort the signal characteristics

used for classification, potentially impacting detection performance. Although the framework incorporates Gaussian noise during validation to simulate some of these effects, further work could explore adaptive preprocessing techniques or trainable noise-resilient encoders to improve robustness under field conditions.

Moreover, although the framework generalizes well across several AES-based hardware Trojan benchmarks, it remains important to consider its adaptability to novel or covert Trojans that may not conform to previously observed patterns. This is especially relevant in rapidly evolving threat environments where attack strategies may be deliberately crafted to evade detection. To enhance resilience in such scenarios, future extensions could include synthetic Trojan generation to diversify training data or meta-learning approaches that facilitate rapid adaptation to new attack profiles.

These considerations highlight opportunities for improving deployment robustness while reinforcing the practical relevance of the proposed method in real-world security-critical systems.

6 Conclusion

This paper has presented a novel approach for detecting Hardware Trojans (HTs) in chip devices, utilizing a combination of Markov Transition Field (MTF) transformations and deep learning techniques. By transforming side-channel signals into image-like formats through MTF, we employed convolutional neural networks (CNNs) to effectively address the intricate task of HT detection. Our evaluation, using the Advanced Encryption Standard (AES) datasets from TrustHub and IEEE Dataport, demonstrated the superior accuracy and classification precision of our method over existing techniques. The proposed approach, characterized by its robust feature extraction and generalization capabilities, achieved an accuracy of approximately 95.2% for AES-T600 and 77.5% for AES-T1600, surpassing previous benchmarks.

Monte Carlo Dropout (MCD) was employed to enhance the model's robustness and reliability by incorporating uncertainty quantification into the decision-making process. Specifically, the model used the standard deviation of predictions generated by MCD to decide whether to accept or reject a prediction, flagging predictions with high uncertainty to reduce erroneous classifications. This approach was particularly effective in benchmarks like AES-T600 and AES-T1600, where reducing the standard deviation threshold improved accuracy, demonstrating that focusing on confident predictions yields higher accuracy. Moreover, MCD allowed dynamic balancing between detection sensitivity and specificity by adjusting the acceptance threshold based on uncertainty. The robustness of the proposed method with

MCD was further highlighted in scenarios involving noise. While the HT detector showed some resilience to moderate noise levels, MCD significantly improved performance by maintaining stable validation accuracy even as noise increased, unlike the non-MCD approach, which declined under high noise levels. This demonstrates MCD's ability to enhance noise resilience and ensure reliable hardware Trojan detection under uncertain conditions.

This study demonstrates the efficacy of combining MTF transformations with deep learning and MCD for HT identification. Future work will focus on enhancing noise robustness and developing models that generalize across a wider range of datasets, improving the identification of various types of HT attacks with even greater accuracy.

Funding Open Access funding provided by the Qatar National Library. This research was supported by NPRP grant NPRP14S-0413-210206 from the Qatar National Research Fund (a member of the Qatar Foundation). Open Access funding was provided by the Qatar National Library.

Declarations

Competing interests The authors declare no competing interests relevant to this work.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Chakraborty, R. S., Narasimhan, S., Bhunia, S.: "Hardware Trojan: Threats and emerging solutions," in 2009 IEEE International High Level Design Validation and Test Workshop. IEEE, nov (2009), pp. 166–171
- Sun, S., Zhang, H., Cui, X., Dong, L., Fang, X.: Electromagnetic Side-Channel Hardware Trojan Detection Based on Transfer Learning. *IEEE Transactions on Circuits and Systems II: Express Briefs* **69**(3), 1742–1746 (2022)
- Tehranipoor, M., Koushanfar, F.: A survey of hardware trojan taxonomy and detection. *IEEE Design and Test of Computers* **27**(1), 10–25 (2010)
- Franco, J., Frick, F.: "Introduction to Hardware Trojan Detection Methods," in Design, Automation & Test in Europe Conference & Exhibition (DATE). New Jersey: IEEE Conference Publications **2015**, 770–775 (2015)
- Xue, M., Gu, C., Liu, W., Yu, S., O'Neill, M.: Ten years of hardware Trojans: a survey from the attacker's perspective. *IET Computers & Digital Techniques* **14**(6), 231–246 (2020)
- Vaddi, E., Gaddam, K., Maniam, R. K., Mallavajjala, S. A., Dasari, S., N. D. M., "Detection and Diagnosis of Hardware Trojan Using Power Analysis," in Security in Computing and Communications, (2015), pp. 519–529
- Pan, Z., Mishra, P.: "A Survey on Hardware Vulnerability Analysis Using Machine Learning," *IEEE Access*, vol. 10, pp. 49 508–49 527, (2022)
- Wang, H., Panoff, M. K., Wang, S., Forte, D.: "HT-EMIS: A Deep Learning Tool for Hardware Trojan Detection and Identification through Runtime EM Side-Channels," in Proceedings of the Great Lakes Symposium on VLSI 2023. New York, NY, USA: ACM, jun 2023, pp. 51–56
- Chen, S., Wang, T., Huang, Z., Hou, X.: Detection method of Golden Chip-Free Hardware Trojan based on the combination of ResNeXt structure and attention mechanism. *Computers and Security* **134**(0123456789), 103428 (2023)
- Mukhtar, N., Fournaris, A. P., Khan, T. M., Dimopoulos, C., Kong, Y.: "Improved hybrid approach for side-channel analysis using efficient convolutional neural network and dimensionality reduction," *IEEE Access*, vol. 8, no. M1, pp. 184 298–184 311, (2020)
- Kaviani, S., Sohn, I.: Defense against neural trojan attacks: A survey. *Neurocomputing* **423**, 651–667 (2021)
- Wang, Z., Oates, T.: "Encoding time series as images for visual inspection and classification using tiled convolutional neural networks," *AAAI Workshop - Technical Report*, vol. WS-15-14, pp. 40–46, (2015)
- Ahmad, Z., Khan, N.: "Inertial sensor data to image encoding for human action recognition," *IEEE Sensors Journal*, vol. 21, no. 9, pp. 10 978–10 988, (2021)
- Barra, S., Carta, S.M., Corrigan, A., Podda, A.S., Recupero, D.R.: Deep learning and time series-to-image encoding for financial forecasting. *IEEE/CAA Journal of Automatica Sinica* **7**(3), 683–692 (2020)
- Gal, Y., Ghahramani, Z.: "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," 33rd International Conference on Machine Learning, ICML 2016, vol. 3, pp. 1651–1660, (2016)
- Bhunia, S., Hsiao, M.S., Banga, M., Narasimhan, S.: Hardware trojan attacks: Threat analysis and countermeasures. *Proceedings of the IEEE* **102**(8), 1229–1247 (2014)
- Vosatka, J.: "Introduction to Hardware Trojans," in The Hardware Trojan War, pp. 15–51. Springer International Publishing, Cham (2018)
- Jacob, N., Merli, D., Heyszl, J., Sigl, G.: Hardware Trojans: current challenges and approaches. *IET Computers & Digital Techniques* **8**(6), 264–273 (2014)
- Liakos, K.G., Georgakilas, G.K., Moustakidis, S., Sklavos, N., Plessas, F.C.: "Conventional and machine learning approaches as countermeasures against hardware trojan attacks," *Microprocessors and Microsystems*, vol. 79, no. September, p. 103295, (2020)
- Naveenkumar, R., Sivamangai, N.M.: Hardware Trojans Detection and Prevention Techniques Review. *Wireless Personal Communications* **136**(2), 1147–1182 (2024)
- Yang, Y., Ye, J., Cao, Y., Zhang, J., Li, X., Li, H., Hu, Y.: "Survey: Hardware Trojan Detection for Netlist," *Proceedings of the Asian Test Symposium*, vol. 2020-Novem, pp. 9–14, (2020)
- Mukherjee, R., Rajendran, S.R., Chakraborty, R.S.: A comprehensive survey of physical and logic testing techniques for Hardware Trojan detection and prevention. *Journal of Cryptographic Engineering* **12**(4), 495–522 (2022)
- Govindan, V., Chakraborty, R.S.: "Logic Testing for Hardware Trojan Detection," in The Hardware Trojan War, pp. 149–182. Springer International Publishing, Cham (2018)
- Flottes, M.-L., Dupuis, S., Ba, P.-S., Rouzeyre, B.: "On the limitations of logic testing for detecting Hardware Trojans Horses," in 2015 10th International Conference on Design & Technology

- of *Integrated Systems in Nanoscale Era (DTIS)*. IEEE, apr (2015), pp. 1–5
25. Rooney, C., Seeam, A., Bellekens, X.: Creation and detection of hardware trojans using non-invasive off-the-shelf technologies. *Electronics (Switzerland)* **7**(7), 1–21 (2018)
 26. Vincent, H., Wells, L., Tarazaga, P., Camelio, J.: Trojan Detection and Side-channel Analyses for Cyber-security in Cyber-physical Manufacturing Systems. *Procedia Manufacturing* **1**, 77–85 (2015)
 27. Narasimhan, S., Du, D., Chakraborty, R.S., Paul, S., Wolff, F.G., Papachristou, C.A., Roy, K., Bhunia, S.: Hardware trojan detection by multiple-parameter side-channel analysis. *IEEE Transactions on Computers* **62**(11), 2183–2195 (2013)
 28. Gubbi, K.I., Saber Latibari, B., Srikanth, A., Sheaves, T., Beheshti-Shirazi, S.A., Pd, S.M., Rafatirad, S., Sasan, A., Homayoun, H., Salehi, S.: “Hardware Trojan Detection Using Machine Learning: A Tutorial,” *ACM Transactions on Embedded Computing Systems*, **22**, 3, (2023)
 29. Huang, Z., Wang, Q., Chen, Y., Jiang, X.: “A Survey on Machine Learning against Hardware Trojan Attacks: Recent Advances and Challenges,” *IEEE Access*, vol. 8, pp. 10 796–10 826, (2020)
 30. Koylu, T. C., Wedig Reinbrecht, C. R., Gebregiorgis, A., Hamdioui, S., Taouil, M.: “A Survey on Machine Learning in Hardware Security,” *ACM Journal on Emerging Technologies in Computing Systems*, vol. 19, no. 2, (2023)
 31. Michailidis, E. T., Kogias, D. G., Voyiatzis, I.: “A Review on Hardware Security Countermeasures for IoT: Emerging Mechanisms and Machine Learning Solutions,” *ACM International Conference Proceeding Series*, no. M1, pp. 268–271, (2020)
 32. Dong, C., Liu, Y., Chen, J., Liu, X., Guo, W., Chen, Y.: “An unsupervised detection approach for hardware trojans,” *IEEE Access*, vol. 8, no. 1c, pp. 158 169–158 183, (2020)
 33. Jap, D., He, W., Bhasin, S.: “Supervised and unsupervised machine learning for side-channel based Trojan detection,” *Proceedings of the International Conference on Application-Specific Systems, Architectures and Processors*, vol. 2016-Novem, pp. 17–24, (2016)
 34. Hoang, T. T., Tran, T. H., Hoang, V. P., Tran, X. N., Pham, C. K.: “Hardware trojan detection techniques using side-channel analysis,” *Proceedings - 2019 6th NAFOSTED Conference on Information and Computer Science, NICS 2019*, pp. 528–533, (2019)
 35. Lodhi, F. K., Hasan, S. R., Hasan, O., Awwad, F.: “Power profiling of microcontroller’s instruction set for runtime hardware Trojans detection without golden circuit models,” *Proceedings of the 2017 Design, Automation and Test in Europe, DATE 2017*, pp. 294–297, (2017)
 36. Mohammed, H., Odetola, T. A., Hasan, S. R., Stissi, S., Garlin, I., Awwad, F.: “(HIADIoT): Hardware Intrinsic Attack Detection in Internet of Things; Leveraging Power Profiling,” *Midwest Symposium on Circuits and Systems*, vol. 2019-Augus, pp. 852–855, (2019)
 37. Dhar, T., Das, R., Giri, C., Roy, S.K.: Threshold Analysis Using Probabilistic Xgboost Classifier for Hardware Trojan Detection. *Journal of Electronic Testing: Theory and Applications (JETTA)* **39**(4), 447–463 (2023)
 38. Bhatta, N. P., Amsaad, F.: “MI assisted techniques in power side channel analysis for trojan classification,” *Cluster Computing* **3**(157), 28 (2025)
 39. Rathor, V.S., Podder, S., Dubey, S.: “An ensemble learning model for hardware Trojan detection in integrated circuit design,” *Computers and Electrical Engineering* **123**(PB), 110090 (2025)
 40. Sagu, A., Gill, N.S., Gulia, P.: Hybrid Deep Neural Network Model for Detection of Security Attacks in IoT Enabled Environment. *International Journal of Advanced Computer Science and Applications* **13**(1), 120–127 (2022)
 41. Rahimifar, M., Jahanirad, H., Fathi, M.: Deep transfer learning approach for digital circuits vulnerability analysis. *Expert Systems with Applications* **237**, 121757 (2024)
 42. Tang, W., Su, J., He, J., Gao, Y.: “A Deep Learning Method Based on the Attention Mechanism for Hardware Trojan Detection,” *Electronics (Switzerland)*, **11**, 15, (2022)
 43. Dakhale, B., Vipinkumar, K., Naroatham, K., Kadam, S., Bhurane, A. A., Kothari, A. G.: “Automated Detection of Hardware Trojans using Power Side-Channel Analysis and VGG-Net,” *2023 2nd International Conference on Paradigm Shifts in Communications Embedded Systems, Machine Learning and Signal Processing, PCEMS 2023*, pp. 1–5, (2023)
 44. Golabi, A., Erradi, A., Bensaid, A., Al-Ali, A., Qidwai, U.: “Enhancing Hardware Trojan Detection: A Dual-Path CNN Approach to Side-Channel Analysis,” in *2024 International Conference on Microelectronics (ICM)*. IEEE, (dec 2024), 1–6
 45. Zhou, F., Zhou, H., Yang, Z., Gu, L.: If2cnn: Towards non-stationary time series feature extraction by integrating iterative filtering and convolutional neural networks. *Expert Systems with Applications* **170**, 114527 (2021)
 46. Semenoglou, A.-A., Spiliotis, E., Assimakopoulos, V.: Image-based time series forecasting: A deep convolutional neural network approach. *Neural Networks* **157**, 39–53 (2023)
 47. Yang, C.-L., Chen, Z.-X., Yang, C.-Y.: Sensor Classification Using Convolutional Neural Network by Encoding Multivariate Time Series as Two-Dimensional Colored Images. *Sensors* **20**(1), 168 (2019)
 48. Ben Said, A., Erradi, A.: “Deep-Gap: A Deep Learning Framework for Forecasting Crowdsourcing Supply-Demand Gap Based on Imaging Time Series and Residual Learning,” in *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, (dec 2019), pp. 279–286
 49. Raubitsek, S., Neubauer, T.: A fractal interpolation approach to improve neural network predictions for difficult time series data. *Expert Systems with Applications* **169**, 114474 (2021)
 50. Jiang, W., Zhang, D., Ling, L., Lin, R.: Time Series Classification Based on Image Transformation Using Feature Fusion Strategy. *Neural Processing Letters* **54**(5), 3727–3748 (2022)
 51. Wu, Y., Wang, B., Yuan, R., Watada, J.: A Gramian angular field-based data-driven approach for multiregion and multisource renewable scenario generation. *Information Sciences* **619**, 578–602 (2023)
 52. Wang, Z., Oates, T.: “Imaging time-series to improve classification and imputation,” *IJCAI International Joint Conference on Artificial Intelligence*, vol. 2015-January, no. Ijcai, pp. 3939–3945, (2015)
 53. Ben Said, A., Abdel-Salam, A.-S. G., Hazaa, K. A.: “Performance prediction in online academic course: a deep learning approach with time series imaging,” *Multimedia Tools and Applications*, (nov 2023)
 54. Barra, S., Carta, S.M., Corriga, A., Podda, A.S., Recupero, D.R.: Deep learning and time series-to-image encoding for financial forecasting. *IEEE/CAA Journal of Automatica Sinica* **7**(3), 683–692 (2020)
 55. Fan, J., Wen, J., Lai, Z.: Myoelectric Pattern Recognition Using Gramian Angular Field and Convolutional Neural Networks for Muscle-Computer Interface. *Sensors* **23**(5), 2715 (2023)
 56. Chandar, S.K.: Convolutional neural network for stock trading using technical indicators. *Automated Software Engineering* **29**(1), 16 (2022)
 57. Tun, W., Wong, K.-W. J., Ling, S.-H.: “Advancing Fault Detection in HVAC Systems: Unifying Gramian Angular Field and 2D Deep Convolutional Neural Networks for Enhanced Performance,” *Sensors* **23**(18), 7690 (2023)

58. Yasaei, R., Faezi, S., Abdullah, M., Faruque, A.: "Power and Electromagnetic Side-Channel Signals of Hardware Trojan Benchmarks," *IEEEDataPort*, (2022)
59. Faezi, S., Yasaei, R., Barua, A., Faruque, M.A.A.: Brain-Inspired Golden Chip Free Hardware Trojan Detection. *IEEE Transactions on Information Forensics and Security* **16**, 2697–2708 (2021)
60. Hu, T., Wu, L., Zhang, X., Liao, Z.: "Hardware Trojan Detection Combines with Machine Learning: An Isolation Forest-based Detection Method," *Proceedings - 2020 IEEE 14th International Conference on Big Data Science and Engineering, BigDataSE 2020*, pp. 96–103, (2020)
61. Zhao, Zhixun, Wei, Pei, Chen, Jihua, Li, Shaoqing, Ni, Lin, "The Influence on Sensitivity of Hardware Trojans Detection by Test Vector," in: *Communications Security Conference (CSC 2014)*. Institution of Engineering and Technology **2014**, 31–31 (2014)
62. Reece, T., Robinson, W.H.: "Analysis of data-leak hardware Trojans in AES cryptographic circuits," *2013 IEEE International Conference on Technologies for Homeland Security, HST 2013*, pp. 467–472, (2013)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.